

String

Word Combinations

```
#include

using namespace std;
typedef long long ll;
const int maxN = 5005, K = 26;
const ll MOD = 1e9+7;

struct Node {
    int next[K];
    bool leaf = false;
    Node(){ fill(next, next+K, -1); }
};

int N, M;
ll dp[maxN];
char S[maxN], word[maxN];
vector<Node> trie(1);

void add_word(){
    scanf("%s", word);
    int v = 0;
    M = (int) strlen(word);
    for(int i = 0; i < M; i++){
        int c = (int) (word[i] - 'a');
        if(trie[v].next[c] == -1){
            trie[v].next[c] = trie.size();
            trie.emplace_back();
        }
        v = trie[v].next[c];
    }
    trie[v].leaf = true;
}

int main(){
    scanf("%s %d", S, &N);
    for(int i = 0; i < N; i++)
        add_word();

    M = (int) strlen(S);
    dp[M] = 1;
    for(int i = M-1; i >= 0; i--){
        int v = 0;
        for(int j = i; j < M; j++){
            int c = (int) (S[j] - 'a');
            if(trie[v].next[c] == -1) break;
            v = trie[v].next[c];

            if(trie[v].leaf)
                dp[i] = (dp[i] + dp[j+1]) % MOD;
        }
    }
    printf("%lld\n", dp[0]);
}
```

String Matching

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e6+5, numP = 3;
const ll MOD = 1e9+7;
const ll prime[numP] = {31, 37, 43};

int N, M, cnt;
ll sh[numP], th[numP], pw[numP][maxN];
char S[maxN], T[maxN];

bool hashes_equal(){
    for(int p = 0; p < numP; p++){
        if(sh[p] != th[p])
            return false;
        return true;
    }
}

int main(){
    scanf("%s %s", S, T);
    N = (int) strlen(S); M = (int) strlen(T);

    if(M > N){
        printf("0\n");
        return 0;
    }

    for(int p = 0; p < numP; p++){
        pw[p][0] = 1;
        for(int i = 1; i < maxN; i++){
            pw[p][i] = (pw[p][i-1] * prime[p]) % MOD;
        }

        for(int i = 0; i < M; i++){
            for(int p = 0; p < numP; p++){
                sh[p] = (sh[p] + (S[i] - 'a' + 1) * pw[p][M-i-1]) % MOD;
                th[p] = (th[p] + (T[i] - 'a' + 1) * pw[p][M-i-1]) % MOD;
            }
        }

        if(hashes_equal()) cnt++;
        for(int i = M; i < N; i++){
            for(int p = 0; p < numP; p++){
                sh[p] = (sh[p] * prime[p] - (S[i-M] - 'a' + 1) * pw[p][M]) % MOD;
                sh[p] = (sh[p] + (S[i] - 'a' + 1) + MOD) % MOD;
            }
            if(hashes_equal()) cnt++;
        }
        printf("%d\n", cnt);
    }
}
```

Finding Borders

```
#include

using namespace std;
```

```

typedef long long ll;
const ll MOD = 1e9+7;
const ll p1 = 31;
const ll p2 = 37;
const int maxN = 1e6+5;

int N;
ll pow1[maxN], pow2[maxN], ph1, ph2, sh1, sh2;
char S[maxN];

int main(){
    scanf("%s", S);
    N = (int) strlen(S);

    pow1[0] = pow2[0] = 1;
    for(int i = 1; i < N; i++){
        pow1[i] = (pow1[i-1] * p1) % MOD;
        pow2[i] = (pow2[i-1] * p2) % MOD;
    }

    for(int i = 0; i < N-1; i++){
        int l = (S[i] - 'a' + 1);
        int r = (S[N-i-1] - 'a' + 1);

        ph1 = (ph1 + l * pow1[i]) % MOD;
        ph2 = (ph2 + l * pow2[i]) % MOD;
        sh1 = (sh1 * p1 + r) % MOD;
        sh2 = (sh2 * p2 + r) % MOD;

        if(ph1 == sh1 && ph2 == sh2)
            printf("%d ", i+1);
    }
}

```

Finding Periods

```

#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;
const ll p1 = 31, p2 = 37;
const int maxN = 1e6+5;

int N;
char S[maxN];
ll pow1[maxN], pow2[maxN], h1[maxN], h2[maxN];

void init(){
    pow1[0] = pow2[0] = 1;
    h1[0] = h2[0] = (int) (S[0] - 'a' + 1);
    for(int i = 1; i < maxN; i++){
        int c = (int) (S[i] - 'a' + 1);
        pow1[i] = (pow1[i-1] * p1) % MOD;
        pow2[i] = (pow2[i-1] * p2) % MOD;
        h1[i] = (h1[i-1] * p1 + c) % MOD;
        h2[i] = (h2[i-1] * p2 + c) % MOD;
    }
}

int main(){

```

```

scanf("%s", S);
N = (int) strlen(S);
init();

for(int len = 1; len <= N; len++){
    ll hash1 = h1[len-1], hash2 = h2[len-1];
    for(int i = 0; i < (N/len)-1; i++){
        hash1 = (hash1 * pow1[len] + h1[len-1]) % MOD;
        hash2 = (hash2 * pow2[len] + h2[len-1]) % MOD;
    }
    hash1 = (hash1 * pow1[N%len] + h1[N%len-1]) % MOD;
    hash2 = (hash2 * pow2[N%len] + h2[N%len-1]) % MOD;
    if(hash1 == h1[N-1] && hash2 == h2[N-1]) printf("%d ", len);
}
}

```

Minimal Rotation

```

#include

using namespace std;
const int maxN = 2e6+5;

int N, F[maxN];
char S[maxN];

int main(){
    scanf("%s", S);

    N = (int) strlen(S);
    for(int i = 0; i < N; i++)
        S[i+N] = S[i];

    N = (int) strlen(S);
    fill(F, F+N, -1);

    int k = 0;
    for(int i = 1; i < N; i++){
        char c = S[i];
        int f = F[i-k-1];
        while(f != -1 && c != S[k+f+1]){
            if(c < S[k+f+1])
                k = i-f-1;
            f = F[f];
        }

        if(c != S[k+f+1]){
            if(c < S[k])
                k = i;
            F[i-k] = -1;
        } else F[i-k] = f+1;
    }

    for(int i = 0; i < N/2; i++)
        printf("%c", S[i+k]);
}

```

Longest Palindrome

```
#include

using namespace std;
const int maxN = 1e6+5;

char S[maxN];
int N, bestl, bestr, d1[maxN], d2[maxN];

int main(){
    scanf(" %s", S);
    N = (int) strlen(S);

    for(int i = 0, l = 0, r = -1; i < N; i++){
        int k = (i > r ? 1 : min(d1[l+r-i], r-i+1));
        while(0 <= i-k && i+k < N && S[i-k] == S[i+k]) k++;

        if(2*k-1 > bestr-bestl+1){
            bestl = i-k+1;
            bestr = i+k-1;
        }

        d1[i] = k--;
        if(i+k > r){
            l = i-k;
            r = i+k;
        }
    }

    for(int i = 0, l = 0, r = -1; i < N; i++){
        int k = (i > r ? 0 : min(d2[l+r-i+1], r-i+1));
        while(0 <= i-k-1 && i+k < N && S[i-k-1] == S[i+k]) k++;

        if(2*k > bestr-bestl+1){
            bestl = i-k;
            bestr = i+k-1;
        }

        d2[i] = k--;
        if(i+k > r){
            l = i-k-1;
            r = i+k;
        }
    }

    for(int i = bestl; i <= bestr; i++)
        printf("%c", S[i]);
}
```

All Palindromes

```
#include
using namespace std;

struct Node {
    int len;           // length of palindrome
    int link;          // suffix link
    array next; // transitions by char
}
```

```

Node(int l = 0) : len(l), link(0) { next.fill(-1); }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    if (!(cin >> s))
        return 0;
    int n = (int)s.size();

    vector tree;
    tree.reserve(n + 5);
    tree.emplace_back(-1); // node 0: length -1 (imaginary)
    tree.emplace_back(0); // node 1: length 0 (empty)
    tree[0].link = 0;
    tree[1].link = 0;

    int last = 1; // index of node representing longest suffix-palindrome so far
    vector ans(n);

    for (int pos = 0; pos < n; ++pos) {
        int c = s[pos] - 'a';
        int cur = last;
        // find largest suffix-palindrome of the current processed prefix that can
        // be extended
        while (true) {
            int curlen = tree[cur].len;
            if (pos - curlen - 1 >= 0 && s[pos - curlen - 1] == s[pos])
                break;
            cur = tree[cur].link;
        }

        if (tree[cur].next[c] != -1) {
            // palindrome already exists
            last = tree[cur].next[c];
        } else {
            // create new node
            int newIdx = (int)tree.size();
            tree.emplace_back(tree[cur].len + 2);
            tree[cur].next[c] = newIdx;

            if (tree[newIdx].len == 1) {
                // single character palindrome -> link to empty palindrome node
                tree[newIdx].link = 1;
            } else {
                // find suffix link for the new node
                int tmp = tree[cur].link;
                while (true) {
                    int tmpLen = tree[tmp].len;
                    if (pos - tmpLen - 1 >= 0 && s[pos - tmpLen - 1] == s[pos])
                        break;
                    tmp = tree[tmp].link;
                }
                tree[newIdx].link = tree[tmp].next[c];
            }
            last = newIdx;
        }

        ans[pos] = tree[last].len;
    }

    for (int i = 0; i < n; ++i) {
        if (i)
            cout << ' ';
        cout << ans[i];
    }
    cout << '\n';
}

```

```
    return 0;
}
```

Required Substring

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1005, maxM = 505;
const ll MOD = 1e9+7;

int N, M, best[26][maxM];
char S[maxN];
ll ans, dp[maxM][maxN];

ll pow26(ll b){
    ll a = 26;
    ll res = 1;
    while(b){
        if(b&1) res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

bool good(vector pre, int k){
    int SZ = (int) pre.size();
    for(int i = 0; i < SZ-k; i++){
        if(pre[i+k] != S[i])
            return false;
    }
    return true;
}

int main(){
    scanf("%d %s", &N, S);
    M = (int) strlen(S);
    if(M > N){
        printf("0\n");
        return 0;
    }

    for(int r = 0; r < M; r++){
        for(int c = 0; c < 26; c++){
            vector pre;
            for(int i = 0; i < r; i++){
                pre.push_back(S[i]);
                pre.push_back((char) (c+'A'));
            }

            for(int k = 0; k < r+1; k++){
                if(good(pre, k)){
                    best[c][r] = r-k+1;
                    break;
                }
            }
        }
    }

    dp[0][0] = 1;
    for(int i = 1; i <= N; i++)
```

```

    for(int j = 0; j < M; j++)
        for(int c = 0; c < 26; c++)
            dp[best[c][j]][i] = (dp[best[c][j]][i] + dp[j][i-1]) % MOD;

ans = pow26(N);
for(int i = 0; i < M; i++)
    ans = (ans - dp[i][N] + MOD) % MOD;
printf("%lld\n", ans);
}

```

Palindrome Queries

```

#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;
const ll p[2] = {31, 37};
const int maxN = 2e5+5;

char S[maxN];
int N, K;
ll pows[2][maxN], invs[2][maxN], F[2], B[2], ds[2][2][maxN];

ll inverse(ll x){
    ll res = 1;
    ll b = MOD-2;
    while(b){
        if(b&1) res = (res * x) % MOD;
        x = (x * x) % MOD;
        b >>= 1;
    }
    return res;
}

void update(int pt, int t, int idx, ll val){
    val %= MOD; val += MOD; val %= MOD;
    for(int i = idx+1; i < maxN; i += -i&i)
        ds[pt][t][i] = (ds[pt][t][i] + val) % MOD;
}

ll query(int pt, int t, int idx){
    if(idx < 0) return 0;
    ll sum = 0;
    for(int i = idx+1; i > 0; i -= -i&i)
        sum = (sum + ds[pt][t][i]) % MOD;
    return sum;
}

void init(){
    for(int t = 0; t < 2; t++){
        pows[t][0] = invs[t][0] = 1;
        for(int i = 1; i < maxN; i++){
            pows[t][i] = (pows[t][i-1] * p[t]) % MOD;
            invs[t][i] = inverse(pows[t][i]);
        }
        update(t, 0, 0, (int) (S[0]-'a'+1) * pows[t][N-1]);
        update(t, 1, 0, (int) (S[0]-'a'+1));
        for(int i = 1; i < N; i++){
            int c = (int) (S[i]-'a'+1);
            update(t, 0, i, c * pows[t][N-i-1]);
        }
    }
}

```

```

        update(t, 1, i, c * pows[t][i]);
    }
}

void modify(int a, char c){
    int newchar = (int) (c - 'a' + 1);
    int oldchar = (int) (S[a] - 'a' + 1);
    for(int t = 0; t < 2; t++){
        update(t, 0, a, -oldchar * pows[t][N-a-1]);
        update(t, 0, a, newchar * pows[t][N-a-1]);
        update(t, 1, a, -oldchar * pows[t][a]);
        update(t, 1, a, newchar * pows[t][a]);
    }
    S[a] = c;
}

bool palindrome(int a, int b){
    for(int t = 0; t < 2; t++){
        F[t] = (query(t, 0, b) - query(t, 0, a-1) + MOD) % MOD;
        F[t] = (F[t] * invs[t][N-b-1]) % MOD;
        B[t] = (query(t, 1, b) - query(t, 1, a-1) + MOD) % MOD;
        B[t] = (B[t] * invs[t][a]) % MOD;
    }
    return F[0] == B[0] && F[1] == B[1];
}

int main(){
    scanf("%d %d %s", &N, &K, S);

    init();
    for(int i = 0, t, a, b; i < K; i++){
        char c;
        scanf("%d", &t);
        if(t == 1){
            scanf("%d %c", &a, &c);
            modify(a-1, c);
        } else if(t == 2){
            scanf("%d %d", &a, &b);
            printf("%s\n", palindrome(a-1, b-1) ? "YES" : "NO");
        }
    }
}

```

Finding Patterns

```

#include

using namespace std;
const int maxN = 1e5+5;

struct Node {
    int len, link, cnt, firstpos;
    map nxt;
} node[2*maxN];

char S[maxN], T[5*maxN];
int N, M, K, sz, last;

void extend(char c){
    int cur = sz++;

```

```

node[cur].len = node[last].len + 1;
int p = last;
while(p != -1 && !node[p].nxt.count(c)){
    node[p].nxt[c] = cur;
    p = node[p].link;
}
if(p == -1){
    node[cur].link = 0;
} else {
    int q = node[p].nxt[c];
    if(node[p].len + 1 == node[q].len){
        node[cur].link = q;
    } else {
        int clone = sz++;
        node[clone].len = node[p].len + 1;
        node[clone].nxt = node[q].nxt;
        node[clone].link = node[q].link;
        while(p != -1 && node[p].nxt[c] == q){
            node[p].nxt[c] = clone;
            p = node[p].link;
        }
        node[q].link = node[cur].link = clone;
    }
}
last = cur;
}

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

bool query(){
    int u = 0;
    for(int i = 0; i < M; i++){
        char c = T[i];
        if(!node[u].nxt.count(c)) return false;
        else u = node[u].nxt[c];
    }
    return true;
}

int main(){
    scanf(" %s %d", S, &K);
    N = (int) strlen(S);
    init();

    for(int i = 0; i < N; i++)
        extend(S[i]);

    for(int i = 0; i < K; i++){
        scanf(" %s", T);
        M = (int) strlen(T);
        printf("%s\n", query() ? "YES" : "NO");
    }
}

```

Counting Patterns

```

#include

using namespace std;
const int maxN = 1e5+5;

struct Node {
    int len, link, cnt;
    map nxt;
} node[2*maxN];

char S[maxN], T[5*maxN];
int N, M, K, sz, last;

void extend(char c){
    int cur = sz++;
    node[cur].cnt = 1;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
    last = cur;
}

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void update_cnts(){
    vector states_by_len[sz];
    for(int i = 0; i < sz; i++){
        states_by_len[node[i].len].push_back(i);
    }
    for(int i = sz-1; i >= 0; i--){
        for(int u : states_by_len[i])
            if(node[u].link != -1)
                node[node[u].link].cnt += node[u].cnt;
    }
}

int query(){
    int u = 0;
    for(int i = 0; i < M; i++){
        char c = T[i];
        if(!node[u].nxt.count(c)) return 0;
        else u = node[u].nxt[c];
    }
    return node[u].cnt;
}

```

```

int main(){
    scanf(" %s %d", S, &K);
    N = (int) strlen(S);

    init();
    for(int i = 0; i < N; i++){
        extend(S[i]);
        update_cnts();
    }

    for(int i = 0; i < K; i++){
        scanf(" %s", T);
        M = (int) strlen(T);
        printf("%d\n", query());
    }
}

```

Pattern Positions

```

#include

using namespace std;
const int maxN = 1e5+5;

struct Node {
    int len, link, cnt, firstpos;
    map<int, int> nxt;
} node[2*maxN];

char S[maxN], T[5*maxN];
int N, M, K, sz, last;

void extend(char c){
    int cur = sz++;
    node[cur].cnt = 1;
    node[cur].firstpos = node[last].len;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            node[clone].firstpos = node[q].firstpos;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
}

```

```

    last = cur;
}

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void update_cnts(){
    vector states_by_len[sz];
    for(int i = 0; i < sz; i++){
        states_by_len[node[i].len].push_back(i);
    }
    for(int i = sz-1; i >= 0; i--){
        for(int u : states_by_len[i])
            if(node[u].link != -1)
                node[node[u].link].cnt += node[u].cnt;
    }
}

int query_cnt(){
    int u = 0;
    for(int i = 0; i < M; i++){
        char c = T[i];
        if(!node[u].nxt.count(c)) return 0;
        else u = node[u].nxt[c];
    }
    return node[u].cnt;
}

int query_idx(){
    int u = 0;
    for(int i = 0; i < M; i++){
        char c = T[i];
        if(!node[u].nxt.count(c)) return -1;
        else u = node[u].nxt[c];
    }
    return node[u].firstpos-M+2;
}

int main(){
    scanf(" %s %d", S, &K);
    N = (int) strlen(S);

    init();
    for(int i = 0; i < N; i++)
        extend(S[i]);
    update_cnts();

    for(int i = 0; i < K; i++){
        scanf(" %s", T);
        M = (int) strlen(T);
        printf("%d\n", query_idx());
    }
}

```

Distinct Substrings

```

#include

using namespace std;

```

```

typedef long long ll;
const int maxN = 1e5+5;

struct Node {
    ll dp;
    int len, link;
    map<int, int> nxt;
} node[2*maxN];

char S[maxN];
int N, sz, last;

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void extend(char c){
    int cur = sz++;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
    last = cur;
}

void calc(int u = 0){
    node[u].dp = 1;
    for(const auto& [c, v] : node[u].nxt){
        if(!node[v].dp) calc(v);
        node[u].dp += node[v].dp;
    }
}

int main(){
    scanf("%s", S);
    N = (int) strlen(S);

    init();
    for(int i = 0; i < N; i++)
        extend(S[i]);
    calc();

    printf("%lld\n", node[0].dp-1);
}

```

Distinct Subsequences

```
#include

using namespace std;
typedef long long ll;
const int maxN = 5e5+5;
const ll MOD = 1e9+7;

int N;
ll tot, dp[26];
char S[maxN];

int main(){
    scanf(" %s", S);
    N = (int) strlen(S);
    for(int i = 0; i < N; i++){
        int c = (int) (S[i] - 'a');
        dp[c]++;
        for(int j = 0; j < 26; j++)
            if(j != c)
                dp[c] = (dp[c] + dp[j]) % MOD;
    }

    for(int i = 0; i < 26; i++)
        tot = (tot + dp[i]) % MOD;
    printf("%lld\n", tot);
}
```

Repeating Substring

```
#include

using namespace std;
const int maxN = 1e5+5;

struct Node {
    int len, link, cnt, firstpos;
    map<int, int> nxt;
} node[2*maxN];

char S[maxN];
bool vis[2*maxN];
int N, sz, last, bestidx, bestlen;

void extend(char c){
    int cur = sz++;
    node[cur].cnt = 1;
    node[cur].firstpos = node[last].len;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
}
```

```

    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            node[clone].firstpos = node[q].firstpos;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
    last = cur;
}

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void update_cnts(){
    vector states_by_len[sz];
    for(int i = 0; i < sz; i++)
        states_by_len[node[i].len].push_back(i);
    for(int i = sz-1; i >= 0; i--)
        for(int u : states_by_len[i])
            if(node[u].link != -1)
                node[node[u].link].cnt += node[u].cnt;
}

void dfs(int u = 0){
    vis[u] = true;
    if(node[u].len > bestlen && node[u].cnt > 1 && u != 0){
        bestidx = node[u].firstpos - node[u].len + 1;
        bestlen = node[u].len;
    }
    for(const auto& [c, v] : node[u].nxt)
        if(!vis[v])
            dfs(v);
}

int main(){
    scanf(" %s", S);
    N = (int) strlen(S);

    init();
    for(int i = 0; i < N; i++)
        extend(S[i]);
    update_cnts();

    bestlen = -1;
    dfs();

    if(bestlen == -1) printf("-1\n");
    else {
        for(int i = 0; i < bestlen; i++)
            printf("%c", S[bestidx+i]);
    }
}

```

String Functions

```
#include

using namespace std;
const int maxN = 1e6+5;

char S[maxN];
int N, z[maxN], pi[maxN];

int main(){
    scanf("%s", S);
    N = (int) strlen(S);

    for(int i = 1, l = 0, r = 0; i < N; i++){
        if(i <= r) z[i] = min(r-i+1, z[i-l]);
        while(i+z[i] < N && S[z[i]] == S[i+z[i]]) z[i]++;
        if(i+z[i]-1 > r) l = i, r = i+z[i]-1;
    }
    for(int i = 0; i < N; i++)
        printf("%d%c", z[i], (" \n")[i==N-1]);

    for(int i = 1; i < N; i++){
        int j = pi[i-1];
        while(j > 0 && S[i] != S[j]) j = pi[j-1];
        if(S[i] == S[j]) j++;
        pi[i] = j;
    }
    for(int i = 0; i < N; i++)
        printf("%d%c", pi[i], (" \n")[i==N-1]);
}
```

String Transform

```
#include

using namespace std;
const int maxN = 1e6+5;

int N, nxt[maxN];
char S[maxN];

int main(){
    scanf("%s", S);
    N = (int) strlen(S);
    for(int i = 0; i < N; i++) nxt[i] = i;
    stable_sort(nxt, nxt+N, [](int a, int b){
        return S[a] < S[b];
    });

    int ptr = nxt[0];
    while(ptr != 0){
        ptr = nxt[ptr];
        printf("%c", S[ptr]);
    }
```

```
    }
}
```

Substring Order I

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+5;

struct Node {
    ll dp;
    int len, link;
    map nxt;
} node[2*maxN];

vector ans;
char S[maxN];
int N, sz, last;
ll K;

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void extend(char c){
    int cur = sz++;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
    last = cur;
}

void calc(int u = 0){
    node[u].dp = 1;
    for(const auto& [c, v] : node[u].nxt){
```

```

        if(!node[v].dp) calc(v);
        node[u].dp += node[v].dp;
    }
}

void dfs(int u, ll k){
    if(k < 0) return;
    for(const auto& [c, v] : node[u].nxt){
        if(node[v].dp <= k) k -= node[v].dp;
        else {
            ans.push_back(c);
            dfs(v, k-1);
            return;
        }
    }
}

int main(){
    scanf("%s %lld", S, &K);
    N = (int) strlen(S);

    init();
    for(int i = 0; i < N; i++)
        extend(S[i]);
    calc();

    dfs(0, K-1);
    int M = (int) ans.size();
    for(int i = 0; i < M; i++)
        printf("%c", ans[i]);
    printf("\n");
}

```

Substring Order II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+5;

struct Node {
    ll dp;
    int len, cnt, link;
    map nxt;
} node[2*maxN];

vector ans;
char S[maxN];
int N, sz, last;
ll K;

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void extend(char c){
    int cur = sz++;

```

```

    node[cur].cnt = 1;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
    last = cur;
}

void update_cnts(){
    vector states_by_len[sz];
    for(int i = 0; i < sz; i++){
        states_by_len[node[i].len].push_back(i);
    }
    for(int i = sz-1; i >= 0; i--){
        for(int u : states_by_len[i])
            if(node[u].link != -1)
                node[node[u].link].cnt += node[u].cnt;
    }
}

void calc(int u = 0){
    node[u].dp = node[u].cnt;
    for(const auto& [c, v] : node[u].nxt){
        if(!node[v].dp) calc(v);
        node[u].dp += node[v].dp;
    }
}

void dfs(int u, ll k){
    if(k < 0) return;
    for(const auto& [c, v] : node[u].nxt){
        if(node[v].dp <= k) k -= node[v].dp;
        else {
            ans.push_back(c);
            dfs(v, k - node[v].dp);
            return;
        }
    }
}

int main(){
    scanf("%s %lld", S, &K);
    N = (int) strlen(S);

    init();
    for(int i = 0; i < N; i++)
        extend(S[i]);
    update_cnts();
    calc();

    dfs(0, K-1);
    int M = (int) ans.size();

```

```

    for(int i = 0; i < M; i++)
        printf("%c", ans[i]);
    printf("\n");
}

```

Substring Distribution

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+5;

struct Node {
    int len, link;
    map nxt;
} node[2*maxN];

char S[maxN];
bool vis[2*maxN];
int N, sz, last, dist[2*maxN];
ll ans[maxN];

void extend(char c){
    int cur = sz++;
    node[cur].len = node[last].len + 1;
    int p = last;
    while(p != -1 && !node[p].nxt.count(c)){
        node[p].nxt[c] = cur;
        p = node[p].link;
    }
    if(p == -1){
        node[cur].link = 0;
    } else {
        int q = node[p].nxt[c];
        if(node[p].len + 1 == node[q].len){
            node[cur].link = q;
        } else {
            int clone = sz++;
            node[clone].len = node[p].len + 1;
            node[clone].nxt = node[q].nxt;
            node[clone].link = node[q].link;
            while(p != -1 && node[p].nxt[c] == q){
                node[p].nxt[c] = clone;
                p = node[p].link;
            }
            node[q].link = node[cur].link = clone;
        }
    }
    last = cur;
}

void init(){
    node[0].len = 0;
    node[0].link = -1;
    sz = 1;
    last = 0;
}

void bfs(int s = 0){
    queue Q;

```

```
vis[s] = true;
dist[s] = 0;
Q.push(s);
while(!Q.empty()){
    int u = Q.front();
    Q.pop();
    ans[dist[u]]++;
    ans[node[u].len+1]--;
    for(const auto& [c, v] : node[u].nxt){
        if(!vis[v]){
            dist[v] = dist[u]+1;
            vis[v] = true;
            Q.push(v);
        }
    }
}

int main(){
    scanf(" %s", S);
    N = (int) strlen(S);
    init();

    for(int i = 0; i < N; i++)
        extend(S[i]);

    bfs();
    for(int i = 1; i <= N; i++){
        ans[i] += ans[i-1];
        printf("%lld%c", ans[i], (" \n")[i==N]);
    }
}
```